

Ssatır satır açıklama:

bash

#!/bin/bash

Shebang satırı — script'in bash yorumlayıcısı ile çalıştırılacağını belirtir.

bash

```
if [ "$(id -u)" -ne 0 ]; then
```

```
    echo "Error: You must run this script as root." >&2
```

```
    exit 1
```

```
fi
```

- `id -u` → çalıştıran kullanıcının UID'sini verir, root'un UID'si 0'dır.
- `-ne 0` → UID 0'a eşit değilse (yani root değilse) blok çalışır.
- `>&2` → hata mesajını stderr'e yönlendirir.
- `exit 1` → script'i hatayla sonlandırır. /etc, /boot, /usr gibi sistem dizinlerini okumak ve /mnt/backup gibi bir yere yazmak genelde root yetkisi gerektirdiği için bu kontrol en başta yapılıyor.

bash

```
sources=(/home/ec2-user/data /etc /boot /usr)
```

Bu bir **bash dizisi (array)** tanımı. Yedeklenecek dört klasörün yolunu tek bir değişkende topluyor. Parantez () içindeki boşlukla ayrılmış değerler dizinin elemanları oluyor.

bash

```
for src in "${sources[@]}"; do
```

```
    if [ ! -e "$src" ]; then
```

```
        echo "Error: $src does not exist. Aborting backup." >&2
```

```
        exit 1
```

```
    fi
```

```
done
```

- `"${sources[@]}"` → dizideki **tüm elemanları**, her biri ayrı bir kelime olarak (aralarında boşluk olsa bile bozulmadan) genişletir. Tırnak içinde `[@]` kullanmak, eleman isimlerinde boşluk olsa bile her elemanı doğru şekilde ayrı ayrı işlemeyi garanti eder — bu bash'te önemli bir best practice.
- `for src in ...; do ... done` → diziyi tek tek dolaşan bir döngü, her elemanı sırayla `src` değişkenine atar.
- `[ ! -e "$src" ]` → `-e` bir dosya/dizin **var mı** diye kontrol eder, `!` ile bunun tersini (yok mu) kontrol ediyoruz.

- Yani: dizideki dört yoldan **herhangi biri sistemde yoksa**, hata basıp script'i durduruyor. Bu, yedekleme sırasında eksik bir klasör yüzünden yarım/bozuk arşiv oluşmasını önüyor.

bash

```
dest="/mnt/backup"
```

```
if [ ! -d "$dest" ]; then
```

```
    echo "Error: destination folder $dest does not exist. Aborting backup." >&2
```

```
    exit 1
```

```
fi
```

- dest → yedeklerin kaydedileceği hedef klasörü tutan değişken.
- -d "\$dest" → bu yolun **bir dizin olarak var olup olmadığını** kontrol eder (dosya değil, klasör olmalı).
- Hedef klasör yoksa, tar komutu zaten hata verirdi ama script burada **önceden kontrol ederek** daha anlaşılır bir hata mesajı veriyor ve gereksiz tar çalıştırmadan çıkıyor.

bash

```
hostname=$(hostname -s)
```

```
day=$(date +%Y%m%d-%H%M)
```

```
archive_file="${hostname}-${day}.tgz"
```

- hostname -s → makinenin **kısa adını** (domain kısmı olmadan, örn. web01) verir. \$(...) ile bu komutun çıktısı hostname değişkenine atanır.
- date +%Y%m%d-%H%M → tarihi istenen formatta üretir:
 - %Y → yıl (4 haneli, örn. 2026)
 - %m → ay (2 haneli)
 - %d → gün (2 haneli)
 - %H → saat (24 saat formatında)
 - %M → dakika
 - Sonuç örneği: 20260702-0715
- archive_file="\${hostname}-\${day}.tgz" → bu iki değeri birleştirip arşiv dosyasının adını oluşturuyor, örn: web01-20260702-0715.tgz. \${...} süslü parantez kullanımı, değişken isminin bitişik metinle (-, .tgz) karışmasını önlemek için kullanılıyor (mesela \$hostname- yazsaydık bash hostname- diye bir değişken arayabilirdi, süslü parantez bunu netleştiriyor).
- .tgz uzantısı, tar.gz (hem arşivlenmiş hem sıkıştırılmış) dosyalar için yaygın kısaltmadır.

bash

```
echo "Backing up ${sources[*]} to $dest/$archive_file"
```

date

echo

- `${sources[*]}` → dizideki tüm elemanları **tek bir string** olarak, aralarına boşluk koyarak birleştirir (ekrana yazdırmak için `[@]` yerine `[*]` kullanmak daha okunaklı bir tek satır çıktı verir).
- `date` → o anki tarih/saati ekrana basar (yedeklemenin ne zaman başladığını loglamak için).
- `echo` (parametresiz) → boş bir satır basar, sadece çıktıyı görsel olarak ayırmak için.

bash

```
tar czf "$dest/$archive_file" "${sources[@]}"
```

Asıl yedekleme komutu burada:

- `tar` → arşivleme aracı.
- `c` → **create**, yeni bir arşiv oluştur.
- `z` → **gzip** ile sıkıştır.
- `f` → çıktının bir **dosyaya** yazılacağını belirtir, hemen ardından dosya adı gelir (`$dest/$archive_file`).
- `"${sources[@]}"` → yedeklenecek dizinlerin listesi, yine tırnaklı `[@]` kullanılarak her biri ayrı argüman olarak `tar`'a veriliyor. Yani tek komutla dört klasör birden tek bir `.tgz` dosyasına paketleniyor.

bash

echo

```
echo "Backup finished:"
```

date

Yedekleme bittikten sonra tekrar boş satır, bitiş mesajı ve o anki tarih/saat basılıyor (yedeklemenin ne kadar sürdüğünü loglardan görebilmek için başlangıç ve bitiş zamanları kaydediliyor).

bash

```
ls -lh "$dest"
```

- `ls -lh` → hedef klasördeki dosyaları **uzun formatta** (`-l`, izinler/tarih/boyut dahil) ve **insan-okur boyut** birimleriyle (`-h`, örn. 15M yerine 15728640 byte) listeler.
- Bu, script çalıştıktan sonra arşiv dosyasının gerçekten oluştuğunu ve boyutunun makul olduğunu (çok küçükse bir sorun olabileceğini) hızlıca görmek için ekleniyor.

bash

```
# -----
```

```
# To set this script for executing every 5 minutes, add the following
```

```
# line with `crontab -e` (running as root, since root privileges are
```

required to read /etc, /boot and /usr):

#

*/5 * * * * /bin/bash /path/to/backup.sh >> /var/log/backup.log 2>&1

Bu kısım **çalışan kod değil**, sadece yorum (comment) satırları. Cron ayarını nasıl yapacağını anlatıyor:

- crontab -e → root'un cron tablosunu düzenleme komutu (script /etc, /boot, /usr okuduğu için root'un crontab'ına eklenmeli).
- */5 * * * * → cron zamanlama ifadesi, 5 alan sırasıyla: dakika, saat, ayın günü, ay, haftanın günü. */5 dakika alanında "her 5 dakikada bir" anlamına gelir, diğerleri * yani "her" değerini kabul eder → sonuç: **her 5 dakikada bir** çalışır.
- /bin/bash /path/to/backup.sh → script'in tam yolunu bash ile çalıştırır (cron ortamında \$PATH sınırlı olabileceğinden, hem yorumlayıcıyı hem tam yolu açıkça yazmak güvenli bir pratiktir).
- >> /var/log/backup.log → script'in normal çıktısını (echo, date, ls çıktıları) bir log dosyasına **ekleyerek** (append, üzerine yazmadan) kaydeder.
- 2>&1 → **stderr**'i (hata çıktısı, kanal 2) **stdout**'a (kanal 1) yönlendirir, böylece hem normal çıktı hem hatalar aynı log dosyasına düşer, hiçbir şey kaybolmaz.

Genel akış özeti: root kontrolü → yedeklenecek klasörlerin var olduğunu kontrol et → hedef klasörün var olduğunu kontrol et → hostname+tarix ile benzersiz bir arşiv adı oluştur → tar czf ile tek seferde sıkıştırılmış arşiv oluştur → başlangıç/bitiş zamanlarını logla → sonucu ls -lh ile doğrula. Cron ile her 5 dakikada bir otomatik tetiklenmesi öngörülüyor.